electronics

MDPI

Article

Enhancing Federated Learning in Heterogeneous Internet of Vehicles: A Collaborative Training Approach

Chao Wu [1], Hailong Fan [1], Kan Wang [1] and Puning Zhang [2,*]

[1] China Merchants Testing Vehicle Technology Research Institute Co., Ltd., Chongqing 401329, China
[2] School of Communications and Information Engineering, Chongqing University of Posts and Telecommunications, Chongqing 400065, China
* Correspondence: zhangpn@cqupt.edu.cn

Abstract: The current Internet of Vehicles (IoV) faces significant challenges related to resource heterogeneity, which adversely impacts the convergence speed and accuracy of federated learning models. Existing studies have not adequately addressed the problem of resource-constrained vehicles that slow down the federated learning process particularly under conditions of high mobility. To tackle this issue, we propose a model partition collaborative training mechanism that decomposes training tasks for resource-constrained vehicles while retaining the original data locally. By offloading complex computational tasks to nearby service vehicles, this approach effectively accelerates the slow training speed of resource-limited vehicles. Additionally, we introduce an optimal matching method for collaborative service vehicles. By analyzing common paths and time delays, we match service vehicles with similar routes and superior performance within mobile service vehicle clusters to provide effective collaborative training services. This method maximizes training efficiency and mitigates the negative effects of vehicle mobility on collaborative training. Simulation experiments demonstrate that compared to benchmark methods, our approach reduces the impact of mobility on collaboration, achieving large improvements in the training speed and the convergence time of federated learning.

Keywords: federated learning; heterogeneous Internet of Vehicles; collaborative training

Citation: Wu, C.; Fan, H.; Wang, K.; Zhang, P. Enhancing Federated Learning in Heterogeneous Internet of Vehicles: A Collaborative Training Approach. *Electronics* **2024**, *13*, 3999. https://doi.org/10.3390/electronics13203999

Academic Editor: Mehdi Sookhak

Received: 12 September 2024
Revised: 3 October 2024
Accepted: 10 October 2024
Published: 11 October 2024

1. Introduction

In recent years, the rapid technological advancements in the information age have led to a significant increase in the number of Internet of Things (IoT) devices, resulting in the generation of massive amounts of data [1]. Leveraging these data to offer more intelligent and convenient application services has become a major research focus [2]. As a pivotal IoT scenario, the Internet of Vehicles (IoV) has garnered substantial interest from researchers. Data generated or collected by vehicles, including driving information, fault diagnostics, and traffic conditions, can advance intelligent services such as autonomous driving [3], vehicle fault diagnosis [4], and intelligent transportation systems [5].

Machine learning is a powerful tool for analyzing and processing complex data. However, traditional centralized machine learning methods, which require transferring data to a cloud center for processing, raise serious privacy concerns as vehicle users typically prefer to keep their personal data private [6]. Additionally, the IoV can produce data at a rate of approximately 30.23 GB/hour [7], making centralized data collection for model training impractical and imposing significant burdens on communication networks. Google's federated learning (FL) addresses these issues by enabling collaborative model building across devices while ensuring user privacy and legal compliance [8]. FL enables model training across distributed devices without compromising user data privacy. Each device utilizes its local data to train a local model and then transmits only the model parameter updates to a central server. This server aggregates the updates to generate a

global model. This process avoids the transmission of raw user data, allowing for multi-device collaborative learning while ensuring data privacy and security. It transmits model parameters instead of raw data during the training process, reducing the communication network load.

Applying federated learning to the IoV, where various vehicles collaboratively train complex models without compromising privacy, can significantly benefit intelligent transportation and related services. Yet, challenges remain in the IoV's resource-imbalanced and mobile environment. The heterogeneity of resources, varying computation and communication capabilities among vehicles, lead to inconsistent local training times and model parameter uploads [9]. Slow vehicles can delay federated learning convergence and reduce global model aggregation efficiency. While some studies employ methods such as asynchronous aggregation and client selection to mitigate resource heterogeneity, they have not effectively utilized idle vehicular computing resources, leading to overall low system resource utilization [10].

Furthermore, the high mobility of IoV vehicles results in frequent changes in geographical location, network bandwidth, and communication area [11]. They may disconnect from edge servers frequently, making traditional federated learning methods for static devices unsuitable. Previous approaches have addressed resource heterogeneity by offloading tasks from lagging clients to edge servers, reducing aggregation waiting time [12,13]. However, the impact of vehicle mobility on computational offloading has been underexplored. Vehicles often reside briefly within a roadside unit's coverage, necessitating frequent server switching and interrupting collaborative training, hampering federated learning progress.

Regarding the above issues, a collaborative and efficient federated learning method (FedHC) is proposed for heterogeneous Internet of Vehicles, a model partitioning collaborative training mechanism is designed, and a collaborative vehicle matching method is proposed to achieve efficient federated learning training in resource-heterogeneous Internet of Vehicles. The main contributions can be summarized as follows.

(i) A model partition collaborative training method is proposed. Different from the existing data partitioning collaborative federated learning method, which may cause privacy leakage problems caused by data sharing between collaborative vehicles, based on the idea of model partitions, we propose a collaborative training mechanism to decompose training tasks while ensuring that raw user data remain local. This approach chooses the split layer based on real-time bandwidth, which is adjusted dynamically at each global aggregation round. It delegates complex tasks to high-performance devices, significantly reducing the computing load on resource-constrained vehicles.

(ii) An optimal vehicle matching method is presented. Unlike existing collaborative federated learning mechanisms that only offload part of the model training tasks to the edge server and ignore the idle vehicles nearby, we introduce a method for selecting service vehicles through quantitative analysis of common paths and delays. By choosing vehicles with similar paths and superior performance from mobile service vehicle clusters, this approach maximizes the speed of collaborative training while mitigating the negative effects of vehicle mobility.

2. Related Works

2.1. Heterogeneous Federated Learning

With numerous devices participating in federated learning, device heterogeneity becomes a significant issue, as varying computational powers lead to slower devices hindering overall training efficiency. Existing research has tackled this problem with methods such as client selection, model compression, adaptive aggregation, and joint optimization. To minimize adverse effects, Wu et al. [13] improved federated learning training speed by offloading tasks to other devices, yet their focus on static IoT scenarios lacks applicability in more dynamic environments. Chen et al. [14] introduced an asynchronous aggregation mechanism that allowed each client to finish its local training independently, helping to reduce training delays. Zhou et al. [15] designed an adaptive segmentation-enhanced

asynchronous federated learning (AS-AFL) model designed to boost learning efficiency and reliability in sustainable Intelligent Transportation Systems (ITSs) through a decentralized approach. However, this method may lead to participants using outdated global models, which have been updated multiple times by more powerful devices, potentially hindering convergence and causing information degradation and outdated parameter updates.

Nishio et al. [10] addressed this by gathering device resource information and implementing a greedy algorithm for client selection to maximize local model aggregation per round, effectively enhancing model convergence speed. Nevertheless, this approach may exclude devices with high-quality data but limited computational power. Li et al. [16] classified clients as either fast or slow based on their resource heterogeneity. However, this binary classification does not capture the intricate diversity of device resources in real-world scenarios, possibly overlooking slower clients and negatively affecting the global model's performance on those devices. Huang et al. [17] developed clustered federated learning (CFL) to create tailored models for different client groups and proposed an approach for selecting participating clients within each cluster using active learning.

Xu et al. [18] and Diao et al. [19] introduced heterogeneous model designs, employing varying degrees of model pruning based on device performance to decrease client waiting time. However, pruning can compromise model integrity, negatively impacting global model performance and complicating global aggregation.

2.2. Federated Learning in the IoV

The high mobility inherent in the IoV significantly impacts the federated learning process. As vehicles continuously change location, the fluctuation in network bandwidth and communication coverage leads to brief and unstable interactions between vehicles and roadside units (RSUs) across regions.

Xiao et al. [20] accounted for training delays and energy consumption by selecting participating clients based on data quality, adjusting transmission and CPU power, and modeling channels via TCP/IP to allocate wireless resources efficiently, reducing communication imbalance and instability. Taïk et al. [21] designed a clustered vehicle federated learning framework by grouping vehicles with similar data and geographic locations, appointing cluster heads for aggregation, and dynamically adjusting these groups based on vehicle movement to improve communication stability.

Liu et al. [22] utilized a near-end FL approach for vehicle edge computing, reducing heterogeneity's impact but without considering mobility and wireless link variations. Liang et al. [23] and Yang et al. [24] introduced methods for asynchronous aggregation and communication compression, which enhanced aggregation efficiency and stability despite device heterogeneity and vehicular mobility. Zhou et al. [25] proposed a robust hierarchical federated learning framework named RoHFL, enabling hierarchical federated learning to be effectively applied in the IoV with resistance to poisoning attacks. Liu et al. [26] devised a new architecture to implement federated learning in the IoV environment, reducing the long learning delays caused by bandwidth limitations, computing restrictions, and the unreliable communication resulting from vehicle mobility. To meet the strict latency requirements of vehicular networks, a quantization scheme was employed in [27] to reduce the size of local models before uploading them.

Despite these advances, there remains a gap in research focusing on collaborative training mechanisms among heterogeneous vehicles in mobile environments. The two primary challenges are the reduced efficiency of vehicle–server collaboration due to high vehicle mobility and the underutilization of substantial vehicular computing resources, both of which contribute to decreased system resource efficiency in the heterogeneous IoV.

3. System Model

3.1. Federated Learning Architecture

In response to the challenges associated with resource constraints in the IoV, we propose offloading some training tasks from resource-limited vehicles to other devices. This

approach aims to enhance the model training speed of resource-constrained vehicles and reduce the waiting time for global model aggregation. However, offloading computational tasks in federated learning within the IoV presents several difficulties. Unlike traditional machine learning, federated learning primarily focuses on protecting participants' data privacy. Conventional offloading methods involve transferring client data and tasks to cloud platforms, where computations are performed, and results are returned. However, this approach uploads raw data or complete training models to other devices, risks user information leakage and violates federated learning's core principles. Moreover, offloading tasks solely to servers can create excessive communication and computational burdens, especially when faced with numerous requests, and is unfeasible in areas without RSU coverage. Additionally, the mobility of vehicles leads to constant changes in geographical location, causing frequent switching between RSUs during computation offloading. Each switch necessitates the retransmission of relevant components and data, and some information may not migrate successfully between servers. This can result in restarts of the offloading process, significantly hindering the progress of federated learning.

To address these challenges, we propose a collaborative training architecture that shifts training tasks while keeping the original data intact on local devices. Tasks are allocated to nearby vehicles that have available computing power, allowing them to function as both mobile servers and clients in the collaborative training process. This approach enhances system-wide resource utilization and reduces the computational and communication burden on RSUs. Moreover, it facilitates computation offloading in regions without RSU coverage. Our method includes an optimal matching strategy for collaborative vehicles; by assessing overlapping routes and anticipated delays, we can choose service vehicles from a pool of candidates that demonstrate optimal performance and route alignment with client vehicles. These service vehicles engage in collaborative training while maintaining effective communication within the same travel pathway. This strategy minimizes disruptions in computation offloading caused by vehicle movement, thus enhancing the stability and performance of the collaborative training.

As depicted in Figure 1, in the three-layer federated learning architecture consisting of cloud servers, edge servers, and vehicles, the global model is aggregated on the cloud server, and the local model is iteratively trained on the vehicle side. At the initial moment, the cloud server sends the global model to the vehicle through the edge server. After several rounds of local training, the vehicle forms a local model and reports it to the edge server. The edge server then reports it to the cloud server for global model aggregation, which means that each global aggregation round includes several local training rounds. The FedHC interaction process involves several steps: First, the cloud center server utilizes the client training delay analysis outlined in Section 4.1 to assess participating vehicles with limited resources, specifically those with high training delays. The RSU within the affected vehicle's area broadcasts to the nearby mobile service cluster (comprising candidate service vehicles) and employs the collaborative vehicle optimal matching method described in Section 4.3 to select an appropriate service vehicle and determine the current model split layer. Once matching is completed, the cloud center server dispatches the initial model, and the client vehicle updates its local model through the model partition collaborative training process detailed in Section 3.2. Upon completing training, the updated parameters are uploaded to the server, which performs global aggregation. The server then distributes the updated model for subsequent training rounds, repeating this process until convergence is achieved.

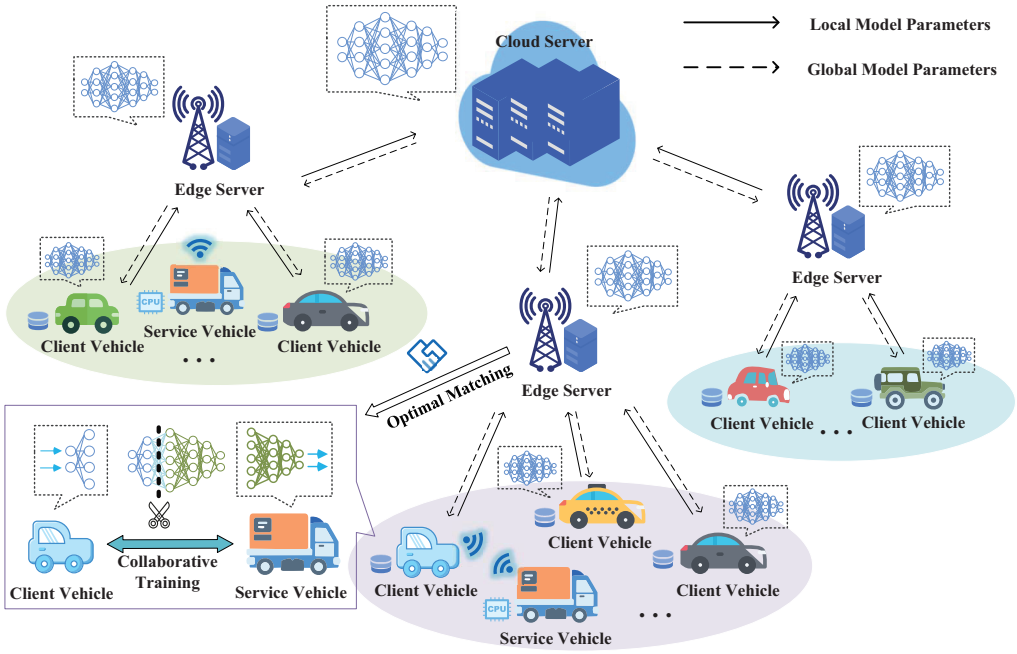


Figure 1. The proposed FedHC framework.

3.2. Model Partition Collaborative Training Process

The proposed collaborative training divides the model into multiple segments and allocates them to collaborative vehicles to complete local model updates jointly. A small number of initial layers, including the input layer, are retained on the client vehicle. Meanwhile, the remaining layers, which require more computations, are offloaded to a service vehicle that provides collaborative computing services. By keeping the initial layers local, the mechanism ensures that the client retains control over the original data, preventing the service vehicle from having knowledge of the complete training model or the original input data. This distribution of computation significantly reduces the client's computational burden and enhances their training speed.

The service vehicle processes the subsequent layers with partial models and intermediate parameters, allowing for collaborative training while ensuring that the client's original data remain on the local device. This approach keeps the complete training model obscured from the service vehicle. Consequently, vehicles with limited computational power can efficiently complete each round of federated learning training. While this method requires additional communication, which introduces slight delays, the significant reduction in training time ultimately accelerates the overall federated learning process.

The FedHC primarily employs two collaborative partition training methods. The first is binary collaborative training, as presented in Figure 2, which involves dividing the training model into two segments. The client is tasked with training the initial layers of the model, while the server handles the latter layers. During training, the client inputs feature data X into its local network and performs forward calculations up to the split layer P, which is the boundary between the two model segments. The output P_{out} at this split layer is transmitted to the server alongside the corresponding label Y. The server then uses this input to continue the forward operation and obtain the predicted label output $\hat{Y}$. $\hat{Y}$ and Y are substituted into the loss function $Loss(\hat{Y},Y)$ for differentiation. The backpropagation algorithm is employed to reverse-transfer the gradient to the preceding layer, calculate the gradient for each layer, and send these results P_{in} back to the client up to the split layer. The client then completes the remaining backpropagation operations, thus finalizing an iterative update. The split layer is determined based on real-time bandwidth and other factors, and this selection is adjusted dynamically at each global aggregation round.

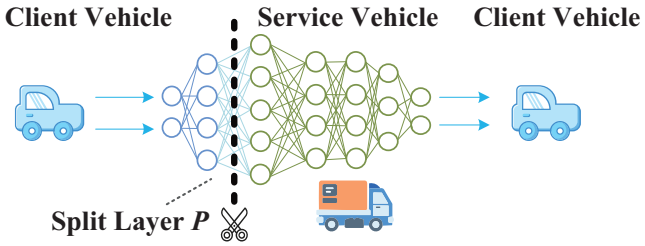


Figure 2. Binary collaborative training.

The aforementioned collaborative partitioning method involves transmitting label information to the server, which poses potential risks of label leakage. To mitigate these risks, we propose an alternative approach: ternary collaborative training, as illustrated in Figure 3. This method partitions the network into three segments: front, middle, and back, with the middle segment comprising the majority of the layers. The client is responsible for training the front and back segments, while the server trains the middle part. The training steps are similar to those in binary collaborative training. However, the key difference is that the final layers are retained locally, eliminating the need to transmit label data to the server and allowing the client to compute the loss function.

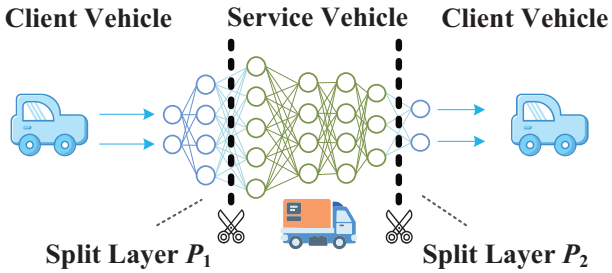


Figure 3. Ternary collaborative training.

Ternary collaborative training enhances security by reducing the exposure of label information. However, this approach requires the client to handle more layers, which can slow down the training speed compared to binary collaborative training. Additionally, since the process involves transmitting data from the client to the server and back to the client, completing one forward operation and backpropagation necessitates two communication transmissions, resulting in greater communication delay. Ternary collaborative training offers improved security but at the cost of slightly slower training speed due to increased communication overhead and local computational demands.

The FedHC system integrates both collaborative methods, leveraging the strengths and addressing the limitations of each. It allows for the selection of the appropriate collaborative method based on the preferences of vehicle users. For instance, clients who have established trust with the server or require higher performance can opt for the binary collaborative training method, which offers faster training speeds. Conversely, clients who prioritize privacy can select the more secure ternary collaborative training approach. This flexibility empowers users to customize settings according to their individual preferences, balancing speed and security based on their specific needs.

4. Optimal Collaborative Vehicle Matching

Given the presence of multiple candidate service vehicles near resource-restricted vehicles, it is crucial to select the optimal service vehicle to provide effective collaborative training services. The selection process involves analyzing both the communication delay and the common path shared by the vehicles. By evaluating these factors, a service vehicle with a longer common path and superior resource can be chosen. This ensures efficient and stable collaborative training by maximizing the overlap in routes and mini-

mizing interruptions, enhancing the overall performance and reliability of the federated learning process.

4.1. Client Latency Analysis

In order to simplify the analysis of the delay in training the local model of client vehicles, we refer to the processing methods of other literature to perform modeling analysis, such as in [28,29]. The total latency experienced by a client participating in training primarily consists of computation latency and transmission latency. This can be analyzed under two scenarios: independent training (without assistance from a service vehicle) and collaborative training.

Independent Training Delay Analysis: This analysis evaluates the performance of the client vehicle to identify those with potential resource constraints, which may require offloading to collaborative training. The independent training delay serves as a benchmark, and the collaborative training delay must be less than this independent delay to justify the offloading process.

Collaborative Training Latency Analysis: By examining collaborative training latency, it is possible to optimize the matching of service vehicles and the selection of the model split layer. Effective analysis ensures that the collaborative training configuration minimizes latency, thus enhancing the efficiency of the training process.

4.1.1. Independent Training Delay

Computation Delay: In a given global aggregation round, the computation delay t_i^{comp} of the local model trained by the client vehicle k_i on its local data samples is

$$t_i^{comp} = \frac{C|D_i|E}{f_i}, \tag{1}$$

where C is the number of CPU cycles required for a local training of a single data sample, $|D_i|$ denotes the number of data samples for vehicle k_i, f_i represents the processor CPU frequency of client vehicle k_i, and E means the number of local training required for a global update of the client.

Transmission Delay: After the client vehicle completes local training, the time required to upload the updated model parameters to the central server can be defined as the size M of the model parameters divided by the transmission rate r_i of the client vehicle k_i. The transmission rate r_i can be expressed as

$$r_i = B\log_2\left(1 + \frac{pw_i g_i}{N_0 B}\right), \tag{2}$$

where B is the total available transmission bandwidth, pw_i is the transmission power of the client vehicle k_i, g_i is the channel gain of the client vehicle k_i, and N_0 is the channel noise power spectral density. The transmission delay t_i^{up} can be defined as

$$t_i^{up} = \frac{M}{r_i} = \frac{M}{B\log_2(1 + \mathrm{pw}_i g_i / N_0 B)}. \tag{3}$$

Total Latency: In each global aggregation round, the total latency t_i required for client vehicle k_i to complete local training and upload can be calculated as

$$t_i = t_i^{comp} + t_i^{up}. \tag{4}$$

4.1.2. Collaborative Training Delay

Next, we analyze the situation where the client vehicle and the service vehicle collaborate to complete local training. The total delay can be divided into calculation delay and transmission delay. The calculation delay includes the training time on the client vehicle side and the training time on the service vehicle side. The transmission delay mainly

includes the transmission delay between the client vehicle and the service vehicle as well as the transmission delay between the client vehicle and the parameter server.

Calculating latency: There is a slight difference in calculating latency between the two collaborative training methods. Firstly, we will discuss binary collaborative training. Since collaborative training involves both client vehicle k_i and service vehicle s_i^j completing the training together, the calculation delay is the sum of client vehicle training time $(E|D_i|C_P^1)/f_i$ and service vehicle training time $(E|D_i|C_P^2)/f_i^j$. It can be formulated as

$$t_{ij}^{comp} = E|D_i|\left(\frac{C_P^1}{f_i} + \frac{C_P^2}{f_i^j}\right), \tag{5}$$

where C_P^1 represents the number of CPU cycles required for the local training of a single data sample in the first part of the client training model after model segmentation through layer P. C_P^2 means the number of CPU cycles required for each iteration of a single sample in the second part of the training model. The sizes of C_P^1 and C_P^2 depend on the selection of the segmentation layer P during model partitioning. Mobile service cluster S_i denotes the set of candidate service vehicles near the geographical location where client vehicle k_i belongs, and f_i^j represents the CPU frequency of the j-th service vehicle s_i^j in mobile service cluster S_i. The formula for calculating latency in ternary collaborative training is similar and can be obtained as

$$t_{ij}^{comp} = E|D_i|\left(\frac{C_P^1 + C_P^3}{f_i} + \frac{C_P^2}{f_i^j}\right). \tag{6}$$

The ternary collaborative training method divides the model into three parts with the client vehicle responsible for training the head and tail of the model; i.e., the client side computation delay can be defined as $E|D_i|(C_P^1 + C_P^3)/f_i$.

Transmission Delay: Similarly, the formula for calculating transmission delay varies under different collaborative training methods. First, let us consider binary collaborative training. During this type of collaborative training, the transmission delay primarily includes two components: the time taken to upload local model parameters to the parameter server after local training and the time for transmitting parameters between the client vehicle and the service vehicle.

The upload delay between the client vehicle and the parameter server is similar to that in Equation (3) with the distinction that in collaborative training, the model parameters are jointly uploaded by the client vehicle and the service vehicle. Each vehicle uploads its respective trained model parameters, which can be expressed as

$$t_{ij}^{up} = \frac{M_P - M_P'}{r_i} + \frac{M_P'}{r_j}, \tag{7}$$

where M_P' represents the storage size of the model parameters trained by the service vehicle, $(M_P - M_P')$ means the parameter size of the partial model trained by the client vehicle, M_P' and M_P depend on the selection of split layer P, and r_j denotes the transmission rate of the service vehicle s_i^j. The transmission delay between the client vehicle and the service vehicle needs to be divided into two situations. If the client vehicle k_i and the service vehicle s_i^j are in the first round of collaborative training, an additional cold start process is required. The client vehicle needs to transmit the server training model to the service vehicle. If it switches to another new service vehicle for collaborative training, it is necessary to send this part of the model again. Otherwise, there is no need to transmit it again.

In addition, during collaborative training, the output P_{out} and input P_{in} of the split layer need to be transmitted between the two vehicles, and the transmission delay $t_{ij}^{trans'}$ between the two vehicles can be expressed as

$$t_{ij}^{trans'} = \begin{cases} \frac{|Model'|}{r_{ij}} + \frac{(|P_{out}| + |P_{in}|)E}{r_{ij}} & \text{if } round_{ij}=1, \\ \frac{(|P_{out}| + |P_{in}|)E}{r_{ij}} & \text{if } round_{ij} \geq 2, \end{cases} \tag{8}$$

where $|P_{out}|$ represents the data storage size of the output result P_{out} of the split layer, $|P_{in}|$ means the data storage size of the input result of the split layer during backpropagation, $|Model'|$ denotes the size of the model transmitted to the server during the cold start process of the first round of collaborative training, r_{ij} is the transmission rate between the two vehicles based on Equation (2), and the transmission delay for cold start is $|Model'|/r_{ij}$. The $round_{ij}$ represents the global aggregation round number of the collaborative training between client vehicle k_i and service vehicle s_i^j.

When $round_{ij} \geq 2$, the transmission time required for each local training iteration is $(|P_{out}| + |P_{in}|)/r_{ij}$, and each global aggregation round requires local training E times, that is, $(|P_{out}| + |P_{in}|)E/r_{ij}$. It can be seen that in order to minimize the transmission delay, switching service vehicles multiple times should be avoided during training. Each switching of a new service vehicle requires an additional cold start (with a cold start time of $|Model'|/r_{ij}$). The transmission delay of the ternary collaborative training method is similar to the above formula, which can be defined as

$$t_{ij}^{trans'} = \begin{cases} \frac{|Model'|}{r_{ij}} + \frac{(|P_{out}| + |P'_{out}| + |P_{in}| + |P'_{in}|)E}{r_{ij}} & \text{if } round_{ij}=1, \\ \frac{(|P_{out}| + |P'_{out}| + |P_{in}| + |P'_{in}|)E}{r_{ij}} & \text{if } round_{ij} \geq 2, \end{cases} \tag{9}$$

where $|P'_{out}|$ and $|P'_{in}|$ are, respectively, the output results of the forward propagation of the second split layer and the input results of the backward propagation. The total transmission delay t_{ij}^{trans} for a single global aggregation round of collaborative training is

$$t_{ij}^{trans} = t_{ij}^{up} + t_{ij}^{trans'}. \tag{10}$$

Collaborative Training Delay: For a single global aggregation round, the total delay t_{ij} of collaborative training for client vehicle k_i can be deduced as

$$t_{ij} = t_{ij}^{comp} + t_{ij}^{trans}, \tag{11}$$

where t_{ij}^{comp} and t_{ij}^{trans} are derived from Equations (5), (6) and (10) depending on the specific situation. When $j = 0$, it represents no corresponding service vehicle, and the client vehicle undergoes independent training.

4.2. Vehicle Common Path Assess

Offloading tasks to nearby vehicles for collaborative training differs significantly from offloading tasks to fixed IoT devices connected to RSUs. The high mobility characteristic of vehicles introduces greater challenges to the IoV. As vehicles move, their positions change constantly, and once the collaborating parties exceed the effective communication range, collaborative training can be interrupted. In such cases, it becomes necessary to either rematch with a new service vehicle or let the client vehicle proceed with independent training. Each switch to a new server requires a cold start, and some collaborative training may not be transferred in time, potentially necessitating a restart of training.

To enhance the efficiency and stability of collaborative training and minimize the disruptions caused by frequent service vehicle switches during task offloading, it is advisable to select service vehicles with similar starting points and longer common driving paths from the candidate mobile service cluster. Service vehicles are encouraged to make partial driving adjustments during collaborative training (by maintaining an effective communication range with the client vehicle while driving) to maximize their service rewards. Short-range broadcasting through RSUs near the client vehicle can be employed to identify service vehicle clusters with approximately the same starting points. The discussion below focuses on selecting service vehicles with longer common driving paths from these mobile service vehicle clusters.

Matching a service vehicle with a longer common path with the client vehicle can reduce the impact of frequent switching among service vehicles during unloading. The most ideal scenario is to maintain the same service vehicle throughout the entire unloading process, and the service vehicle path should include the entire client vehicle path. The vehicle's driving path is obtained by a trusted third party through the navigation information of the two vehicles. The path of client vehicle k_i can be seen as a curve L_i composed of longitude and latitude coordinates, and the path of candidate service vehicles s_i^j near the client vehicle is the curve L_j. d_{ij} represents the common path length between the curve L_j and the curve L_i, which can be formulated as

$$d_{ij} = \left| L_i \cap L_j \right|, \tag{12}$$

where $L_i \bigcap L_j$ represents the intersection of two curves. By sampling L_i as a discrete set of points $\tilde{L}_i$, the common curve problem can be simplified into a common point problem. Specifically, all points in point set $\tilde{L}_i$ that exist in curve L_j are first determined, and then d_{ij} is obtained from these points

$$d_{ij} = \sum_{m=1}^{n-1} |l_{m+1} - l_m|, \forall l_{m+1}, l_m \in (L_i \bigcap L_j), \tag{13}$$

where l_m is a point in the point set $\tilde{L}_i$, $m \in [1, n]$, and n is the number of sampling times (sampling the curve L_i into n points). When two consecutive points l_{m+1}, l_m in the set of points $\tilde{L}_i$ exist in L_j, the distance $|l_{m+1} - l_m|$ between the two points is a part of d_{ij}. Summing all the consecutive two-point distances gives d_{ij}.

4.3. Collaborative Vehicle Match

To enhance the model training speed of resource-constrained vehicles and maximize the efficiency of collaborative training, it is essential to select a service vehicle with superior performance and longer common driving paths from the mobile service clusters. Superior performance is characterized by enhanced computing and communication capabilities. Computational and transmission delays are used to assess these capabilities, while overall delay serves as a measure of the device's overall performance: the lower the delay, the stronger the performance. The objective function for better performance can be calculated as

$$\arg\min_j t_{ij} = \arg\min \ t_{ij}^{comp} + t_{ij}^{trans}. \tag{14}$$

The collaborative computation delay t_{ij}^{comp} and communication delay t_{ij}^{trans} are both related to the actual model split layer P. The split layer needs to be selected based on real-time network bandwidth and other information of the two vehicles after the collaborative vehicle matching is completed. So, before selecting the split layer, the average collaborative training delay is used for delay evaluation in the initial stage of collaborative vehicle

matching. The average collaborative training delay $\bar{t}_{ij}$ is the average training delay of all split layers as

$$\bar{t}_{ij} = \frac{1}{n}\sum_{P=1}^{n} t_{ij} = \frac{1}{n}\sum_{P=1}^{n}(t_{ij}^{comp} + t_{ij}^{trans}), \tag{15}$$

where P is the selection of the split layer with a total of n selection methods. Therefore, the objective function with better performance can be transformed into

$$\arg\min_{j} \bar{t}_{ij} = \arg\min_{j} \frac{1}{n}\sum_{P=1}^{n}(t_{ij}^{comp} + t_{ij}^{trans}), \tag{16}$$

where t_{ij}^{comp} and t_{ij}^{trans} are obtained according to Section 3.2. The goal of a longer common path can be defined as

$$\arg\max_{j} d_{ij} = \arg\max \sum_{m=1}^{n-1} |l_{m+1} - l_m|. \tag{17}$$

A common approach to combining two objective functions is to assign weights to each and sum them, effectively resolving issues related to inconsistent dimensions. However, setting appropriate weights can be challenging and often requires manual adjustment. In practice, it is important to consider how to merge multiple optimization objective functions into a single cohesive function based on the specific problem context. The primary goal of matching collaborative vehicles is to enhance the training speed of the client vehicle through the assistance of service vehicles throughout the entire training period. Thus, the optimization objective can be reframed as maximizing the total number of local model updates sum_i completed within the training timeframe. A higher frequency of updates indicates better collaborative efficiency and a faster overall training speed for the client vehicle.

The overall training speed relates to factors such as the length of the common path, training delay, and driving speed. By focusing on maximizing the total number of updates, the optimization objective effectively integrates the dimensions of common path and training delays, leading to improvements in collaborative training efficiency.

Therefore, the vehicle matching optimization objective can be defined as

$$\arg\max_{j} sum_i(j) = \arg\max \frac{T_{ij}}{\bar{t}_{ij}} + \frac{T_i}{t_i} = \arg\max \frac{d_{ij}}{v_i \bar{t}_{ij}} + \frac{d_i - d_{ij}}{v_i t_i}, \tag{18}$$

$$s.t. \quad t_{ij} < \frac{C|D_i|E}{f_i} + \frac{M}{B\log_2\left(1 + \frac{p_i g_i}{N_0 B}\right)}, \tag{19}$$

$$\frac{d_{ij}}{v_i t_{ij}} + \frac{d_i - d_{ij}}{v_i t_i} > \frac{d_i}{v_i t_i}, \tag{20}$$

where T_{ij} is the duration of collaborative training between client vehicle k_i and service vehicle s_i^j, which is equal to the common driving distance d_{ij} of the two vehicles divided by the vehicle speed v_i (collaborative training is conducted on the common path of the two vehicles). T_i is the duration of independent training for vehicle k_i, which is equal to the independent driving distance $(d_i - d_{ij})$ of vehicle k_i divided by the driving speed v_i. The client vehicle needs to be trained independently outside the common path of the two vehicles, where d_i is the total distance traveled by client vehicle k_i. $\bar{t}_{ij}$ is the average time required for vehicle k_i and s_i^j to complete a collaborative training, as determined by Equations (12) and (16). $T_{ij}/\bar{t}_{ij}$ represents the number of times two vehicles have completed collaborative training, while T_i/t_i represents the number of times vehicle k_i has completed independent training.

The essential purpose of collaborative training is to improve the training speed of client vehicles. Therefore, the collaborative training delay t_{ij} should be not less than the independent training delay, and the total number of training times should be not less than the number of times that vehicle k_i can complete independent training throughout the entire process. Otherwise, vehicle k_i should not match the service vehicle temporarily and will undergo independent local training. So, by comparing the sum_i values of different candidate service vehicles, the service vehicle with the highest sum_i value and meeting the above conditions is selected as the one for collaborative training.

After client vehicle k_i completes the service vehicle matching, the model split layer is selected. Due to the movement of vehicles and other factors, the network bandwidth of vehicles changes in real time. Therefore, the optimal split layer selection may be different for each global aggregation round. Therefore, at each global aggregation round, the training delay of all split layers is evaluated, and the split layer P that minimizes the training delay is selected for this round of collaborative training. This can be formulated as

$$\arg\min_{P} t_{ij}(P) = \arg\min \ t_{ij}^{comp} + t_{ij}^{trans}, \tag{21}$$

where t_{ij}^{comp} and t_{ij}^{trans} can be obtained through Equations (6), (7), (9) and (10), and their values are related to the computing and communication capabilities of the collaborative vehicle as well as the model split layer. Since the collaborative vehicle matching has been completed before the model split layer is selected, the computing and communication capabilities of the collaborative vehicle can be determined at this moment. Therefore, the values of t_{ij}^{comp} and t_{ij}^{trans} in the current training round depend on the selection of the split layer P. Therefore, by comparing the training delay of different split layers P, we can choose the appropriate P, which minimizes the collaborative training delay, as the split layer for the current model of this round.

In summary, through the collaborative vehicle matching method, the currently optimal service vehicle can be matched for a resource-restricted client vehicle. The two vehicles use the V2V (Vehicle to Vehicle) communication mode for collaborative training to complete local model updates. After the local training is completed, the model parameters are uploaded to the server and then aggregated by the server for the next training round.

5. Simulation Result

5.1. Simulation Setup

5.1.1. Models and Datasets

Experiments are conducted using a Convolutional Neural Network (CNN) model on real datasets to evaluate the proposed FedHC method. The model consists of three convolutional layers and three fully connected layers. The datasets used for model training and testing are Fashion-MNIST [30] and CIFAR-10 [31].

Fashion-MNIST: The Fashion-MNIST dataset includes 70,000 grayscale images across 10 categories (such as shirts, pants, shoes, etc.). Out of these, 60,000 images are used for training the model, and 10,000 images are used for testing. Each image has a resolution of 28×28 pixels.

CIFAR-10: The CIFAR-10 dataset comprises 60,000 color images also across 10 categories (including cars, airplanes, cats, dogs, and more). This dataset is divided into 50,000 images for training and 10,000 images for testing with each image having a resolution of 32×32 pixels.

Chongqing Taxi Trajectory Dataset: To simulate vehicle mobility, the Chongqing Taxi Trajectory dataset [32] was used. This dataset provides driving data for taxis at different times, including information on GPS time, latitude and longitude, speed, and direction of travel.

5.1.2. Experimental Environment

The simulation device uses a Linux operating system server with an Intel (R) Core (R) CPU i9 10,900 k @ 3.70 GHz (20 core processor), 64.0 GB of RAM, and a GPU of 2080Ti (11 G RAM). Then, it simulates and validates the proposed method and benchmark method using the Python language. The parameters for the simulation experiments are detailed in Table 1.

Table 1. Simulation parameters.

Parameters	Value
Local training round	5
Global aggregation round	150
Local iterators	SGD
Learning rate	0.01
Batch data volume	32

5.1.3. Resource Heterogeneity

In order to simulate the heterogeneous characteristics of resources in the Internet of Vehicles, virtual machine technology is used to divide server computing resources into several virtual resources, and each vehicle node is allocated a corresponding number of CPU cores, as the same setting in [33]. In the Fashion-MNIST, the number of client nodes is set to 10, and each node is allocated 0.5 to 3 CPU cores, following a normal distribution of $\mu = 1.5$ and $\sigma = 0.7$. In the CIFAR-10, the number of client nodes is set to 8, and each node is allocated 1 to 5 CPU cores, following a normal distribution of $\mu = 3$ and $\sigma = 0.7$.

5.1.4. Benchmark

To validate the performance advantages of the proposed FedHC method, we compared it against three benchmark methods: FedAvg [34], FedAdapt [13], and HFL [35].

FedAvg is a classic federated learning algorithm that involves randomly selecting clients for local training in each round, which is followed by global aggregation and updates on the server.

FedAdapt is an adaptive offloading federated learning algorithm that uses reinforcement learning to adaptively offload client training tasks to the server, enhancing the speed of federated learning training.

HFL is a heterogeneous model federated learning algorithm that adjusts model complexity based on the heterogeneous computational resources of the clients. It prunes models to varying degrees to reduce synchronization delays between clients, improving overall system computational efficiency.

5.1.5. Performance Metrics

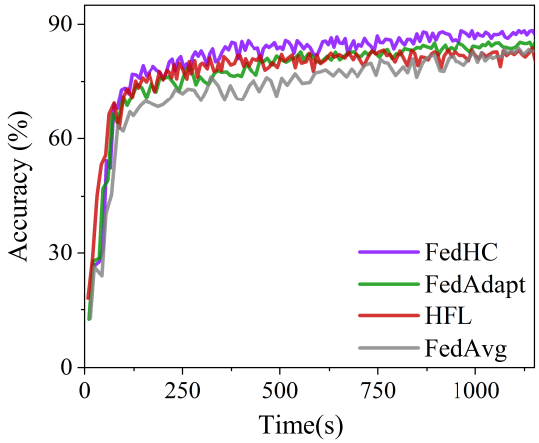
This section evaluates the proposed algorithm using two key metrics: model accuracy and training time.

Model accuracy represents the proportion of correctly classified samples over all samples in the dataset. The global model is tested for accuracy on the server at the end of each training round.

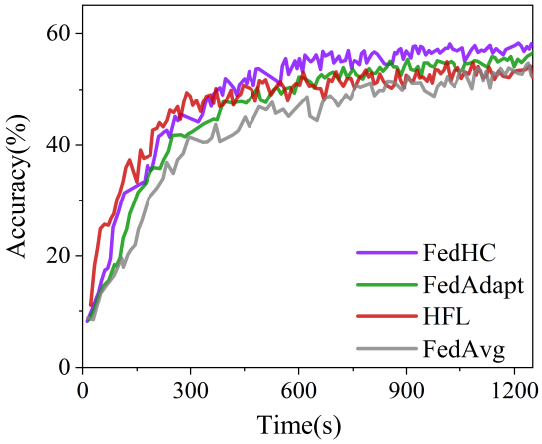
Training time measures the total time consumed until the termination of training and is used to assess the speed of the federated learning process.

5.2. Results and Analysis

Figure 4 illustrates the model accuracy of the proposed FedHC method compared to benchmark methods over the same training time across two datasets. Overall, FedHC achieves the desired accuracy in the least amount of time. By offloading some training tasks to nearby service vehicles, the training speed of resource-constrained vehicle nodes is improved, and the waiting time for aggregation in each global round is significantly reduced.



(**a**) Fashion-MNIST experiment



(**b**) CIFAR-10 experiment

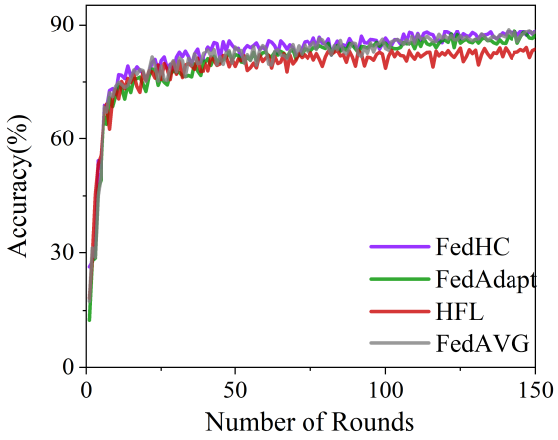
Figure 4. Comparison of test accuracy with the same training time.

The FedAvg method requires the most time to reach the same accuracy compared to HFL, FedHC, and FedAdapt due to its longer waiting times for resource-constrained stragglers during each training round. HFL employs heterogeneous models to reduce client training time, achieving comparable accuracy with less training time in the early stages. However, it underperforms in later stages because model pruning, while reducing training time, results in some accuracy loss, impacting final convergence accuracy.

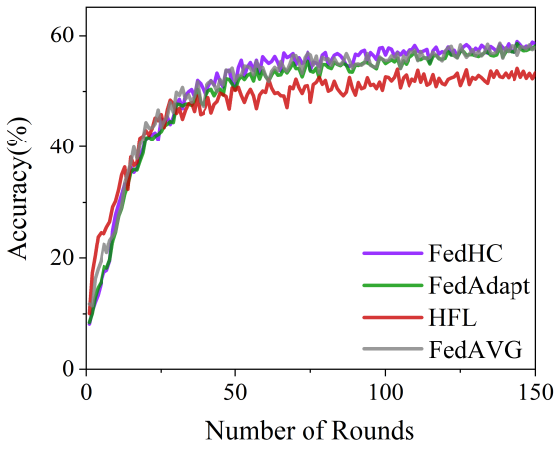
FedAdapt shares a similar approach to FedHC, improving federated learning speed by offloading client training tasks to the server. However, since FedAdapt is designed for static IoT scenarios, its efficiency in the dynamic Internet of Vehicles environment is lower than that of FedHC. The mobility of vehicle nodes leads to frequent switches between RSUs, causing interruptions in computation offloading. In contrast, FedHC effectively matches service vehicles with similar paths, allowing for collaborative efforts where vehicles maintain relative stillness through similar speeds on a common path, thus mitigating the negative effects of mobility on collaborative training.

Figure 5 compares the model accuracy of different methods for the same number of training rounds across two datasets. The FedHC, FedAdapt, and FedAvg methods demonstrate significantly better model accuracy than the HFL method for any given training round. Although HFL enhances model training speed and completes more training rounds in a specific time frame compared to methods like FedAvg, its model compression

adversely affects accuracy. Consequently, HFL results in lower accuracy per training round and reduced final convergence accuracy compared to the other methods. For 150 training rounds on both datasets, the highest accuracies achieved by HFL are 84.02% and 54.11%, respectively. In contrast, FedHC, FedAdapt, and FedAvg achieve similar model accuracies under the same conditions. Specifically, FedHC attains the highest accuracies of 88.46% and 58.87%, FedAdapt achieves 88.55% and 58.49%, and FedAvg reaches 88.29% and 58.55%. These results demonstrate that the proposed FedHC method can enhance training speed without compromising model accuracy.



(**a**) Fashion-MNIST experiment



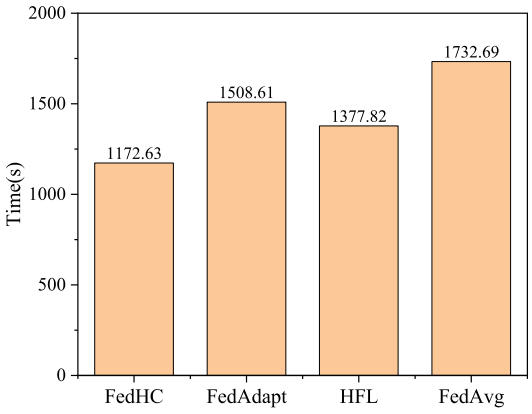
(**b**) CIFAR-10 experiment

Figure 5. Comparison of test accuracy under the same training round.

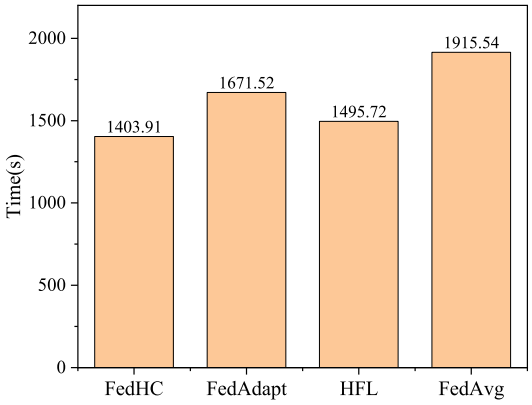
Figure 6 compares the total training time for 150 global aggregation rounds among the benchmark methods across two datasets. For the Fashion-MNIST dataset, the FedAvg method requires approximately 1732.69 s to complete 150 training rounds, while it takes about 1915.54 s for the CIFAR-10 dataset. Among the methods tested, FedAvg has the longest training time, which is largely because each training round is synchronized to the slowest client, necessitating waiting for resource-constrained participants.

Compared to FedAvg, both HFL and FedAdapt demonstrate significant reductions in training time. HFL completes the process in approximately 1377.82 s on the Fashion-MNIST dataset and 1495.72 s on the CIFAR-10 dataset. This reduction is achieved through model compression, which lessens computational complexity and effectively shortens training time. Meanwhile, FedAdapt takes about 1508.61 s and 1671.52 s, respectively, for the two

datasets. However, its efficiency is lower in dynamic Internet of Vehicles scenarios due to unaddressed issues of instabilities caused by mobile nodes during task offloading, resulting in a total training time only slightly lower than that of FedAvg.



(**a**) Fashion-MNIST experiment



(**b**) CIFAR-10 experiment

Figure 6. Compares the total training time of 150 global aggregation rounds.

The FedHC method achieves the shortest training times, requiring roughly 1172.63 s for the Fashion-MNIST dataset and 1403.91 s for the CIFAR-10 dataset. This demonstrates FedHC's capability to optimize training efficiency effectively in a dynamic vehicular environment.

6. Conclusions

In this paper, we address the challenges associated with federated learning in heterogeneous Internet of Vehicles (IoV) environments by introducing a novel collaborative vehicle optimal matching method. Our approach includes a model partition collaborative training method, which delegates complex tasks to high-performance devices, substantially reducing the computational load on resource-constrained vehicles. Additionally, we propose an optimal vehicle-matching method to select service vehicles that maximize the speed of collaborative training while minimizing the adverse effects of vehicle mobility. By pairing constrained client vehicles with suitable service vehicles, our specialized collaborative training mechanism significantly enhances training speed and mitigates the detrimental impact of vehicle mobility on task offloading. Simulation results demonstrate

the effectiveness of our method in improving model accuracy and reducing the training latency in IoV federated learning.

Our research primarily focuses on establishing an efficient vehicle collaborative federated learning mechanism for IoV. We acknowledge, however, that it does not extensively address ensuring reliable communication in complex channel environments or achieving efficient computing between vehicle hardware and software. For future work, exploring the practical deployment of federated learning models in the IoV will be an important area of research. Additionally, we also aim to develop an effective incentive mechanism for collaborative training and continue studying efficient federated learning strategies to further alleviate the impact of resource heterogeneity on federated learning.

Author Contributions: Conceptualization, C.W. and H.F.; methodology, C.W. and K.W.; software, K.W. and P.Z.; validation, C.W. and H.F.; formal analysis, C.W.and P.Z.; investigation, H.F.; resources, C.W.; data curation, P.Z.; writing—original draft preparation, P.Z.; writing—review and editing, C.W. and P.Z.; visualization, K.W.; supervision, C.W.; project administration, C.W.; funding acquisition, C.W. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Science and Technology Innovation Key R&D Program of Chongqing (Grant No. CSTB2022TIADSTX0003), National Key Research and Development Program of China (Grant No.2022YFF0604900), and National Natural Science Foundation of China (Grant No. 62376036).

Data Availability Statement: Data are contained within the article.

Acknowledgments: In this section, we thank all those who contributed to this article.

Conflicts of Interest: Authors Chao Wu, Kan Wang, and Hailong Fan were employed by the company China Merchants Testing Vehicle Technology Research Institute Co., Ltd. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Aouedi, O.; Vu, T.H.; Sacco, A.; Nguyen, D.C.; Piamrat, K.; Marchetto, G.; Pham, Q.V. A survey on intelligent Internet of Things: Applications, security, privacy, and future directions. *IEEE Commun. Surv. Tutor.* **2024**. [CrossRef]
2. Liu, Y.; Wang, J.; Yan, Z.; Wan, Z.; Jäntti, R. A survey on blockchain-based trust management for Internet of Things. *IEEE Internet Things J.* **2023**, *10*, 5898–5922. [CrossRef]
3. Teng, S.; Hu, X.; Deng, P.; Li, B.; Li, Y.; Ai, Y.; Yang, D.; Li, L.; Xuanyuan, Z.; Zhu, F.; et al. Motion planning for autonomous driving: The state of the art and future perspectives. *IEEE Trans. Intell. Veh.* **2023**, *8*, 3692–3711. [CrossRef]
4. Feng, Z.; Yang, R.; Zhou, Z.; Hu, C. Trustworthy Fault diagnosis method based on belief rule base with multisource uncertain information for vehicle. *IEEE Trans. Ind. Electron.* **2023**, *71*, 7947–7956. [CrossRef]
5. Gong, T.; Zhu, L.; Yu, F.R.; Tang, T. Edge intelligence in intelligent transportation systems: A survey. *IEEE Trans. Intell. Transp. Syst.* **2023**, *24*, 8919–8944. [CrossRef]
6. Liang, P.P.; Zadeh, A.; Morency, L.P. Foundations and Trends in Multimodal Machine Learning: Principles, Challenges, and Open Questions. *arXiv* **2022**, arXiv:2209.03430. [CrossRef]
7. Ernest, T.Z.H.; Madhukumar, A. Computation offloading in MEC-enabled IoV networks: Average energy efficiency analysis and learning-based maximization. *IEEE Trans. Mob. Comput.* **2024**, *23*, 6074–6087. [CrossRef]
8. Liu, Y.; Kang, Y.; Zou, T.; Pu, Y.; He, Y.; Ye, X.; Ouyang, Y.; Zhang, Y.Q.; Yang, Q. Vertical federated learning: Concepts, advances, and challenges. *IEEE Trans. Knowl. Data Eng.* **2024**, *36*, 3615–3634. [CrossRef]
9. Hu, X.; Li, R.; Wang, L.; Ning, Y.; Ota, K. A data sharing scheme based on federated learning in iov. *IEEE Trans. Veh. Technol.* **2023**, *72*, 11644–11656. [CrossRef]
10. Nishio, T.; Yonetani, R. Client selection for federated learning with heterogeneous resources in mobile edge. In Proceedings of the ICC 2019-2019 IEEE International Conference on Communications (ICC), Shanghai, China, 20–24 May 2019; pp. 1–7.
11. Maurya, C.; Chaurasiya, V.K. Efficient anonymous batch authentication scheme with conditional privacy in the Internet of Vehicles (IoV) applications. *IEEE Trans. Intell. Transp. Syst.* **2023**, *24*, 9670–9683. [CrossRef]
12. Shen, J.; Wang, X.; Cheng, N.; Ma, L.; Zhou, C.; Zhang, Y. Effectively heterogeneous federated learning: A pairing and split learning based approach. In Proceedings of the GLOBECOM 2023—2023 IEEE Global Communications Conference, Kuala Lumpur, Malaysia, 4–8 December 2023; pp. 5847–5852.
13. Wu, D.; Ullah, R.; Harvey, P.; Kilpatrick, P.; Spence, I.; Varghese, B. Fedadapt: Adaptive offloading for iot devices in federated learning. *IEEE Internet Things J.* **2022**, *9*, 20889–20901. [CrossRef]

14. Chen, Y.; Ning, Y.; Slawski, M.; Rangwala, H. Asynchronous online federated learning for edge devices with non-iid data. In Proceedings of the 2020 IEEE International Conference on Big Data (Big Data), Atlanta, GA, USA, 10–13 December 2020; pp. 15–24.
15. Zhou, X.; Liang, W.; Kawai, A.; Fueda, K.; She, J.; Kevin, I.; Wang, K. Adaptive segmentation enhanced asynchronous federated learning for sustainable intelligent transportation systems. *IEEE Trans. Intell. Transp. Syst.* **2024**, *25*, 6658–6666. [CrossRef]
16. Li, T.; Sahu, A.K.; Zaheer, M.; Sanjabi, M.; Talwalkar, A.; Smith, V. Federated optimization in heterogeneous networks. *Proc. Mach. Learn. Syst.* **2020**, *2*, 429–450.
17. Huang, H.; Shi, W.; Feng, Y.; Niu, C.; Cheng, G.; Huang, J.; Liu, Z. Active client selection for clustered federated learning. *IEEE Trans. Neural Netw. Learn. Syst.* **2023**. [CrossRef]
18. Xu, Z.; Yu, F.; Xiong, J.; Chen, X. Helios: Heterogeneity-aware federated learning with dynamically balanced collaboration. In Proceedings of the 2021 58th ACM/IEEE Design Automation Conference (DAC), San Francisco, CA, USA, 5–9 December 2021; pp. 997–1002.
19. Diao, E.; Ding, J.; Tarokh, V. Heterofl: Computation and communication efficient federated learning for heterogeneous clients. *arXiv* **2020**, arXiv:2010.01264.
20. Xiao, H.; Zhao, J.; Pei, Q.; Feng, J.; Liu, L.; Shi, W. Vehicle selection and resource optimization for federated learning in vehicular edge computing. *IEEE Trans. Intell. Transp. Syst.* **2021**, *23*, 11073–11087. [CrossRef]
21. Taik, A.; Mlika, Z.; Cherkaoui, S. Clustered vehicular federated learning: Process and optimization. *IEEE Trans. Intell. Transp. Syst.* **2022**, *23*, 25371–25383. [CrossRef]
22. Liu, S.; Yu, J.; Deng, X.; Wan, S. FedCPF: An efficient-communication federated learning approach for vehicular edge computing in 6G communication networks. *IEEE Trans. Intell. Transp. Syst.* **2021**, *23*, 1616–1629. [CrossRef]
23. Liang, F.; Yang, Q.; Liu, R.; Wang, J.; Sato, K.; Guo, J. Semi-synchronous federated learning protocol with dynamic aggregation in internet of vehicles. *IEEE Trans. Veh. Technol.* **2022**, *71*, 4677–4691. [CrossRef]
24. Yang, Z.; Zhang, X.; Wu, D.; Wang, R.; Zhang, P.; Wu, Y. Efficient asynchronous federated learning research in the internet of vehicles. *IEEE Internet Things J.* **2022**, *10*, 7737–7748. [CrossRef]
25. Zhou, H.; Zheng, Y.; Huang, H.; Shu, J.; Jia, X. Toward robust hierarchical federated learning in internet of vehicles. *IEEE Trans. Intell. Transp. Syst.* **2023**, *24*, 5600–5614. [CrossRef]
26. Liu, S.; Yu, G.; Yin, R.; Yuan, J.; Qu, F. Communication and computation efficient federated learning for Internet of vehicles with a constrained latency. *IEEE Trans. Veh. Technol.* **2023**, *73*, 1038–1052. [CrossRef]
27. Zhang, X.; Chen, W.; Zhao, H.; Chang, Z.; Han, Z. Joint Accuracy and Latency Optimization for Quantized Federated Learning in Vehicular Networks. *IEEE Internet Things J.* **2024**, *11*, 28876–28890. [CrossRef]
28. Ye, D.; Yu, R.; Pan, M.; Han, Z. Federated learning in vehicular edge computing: A selective model aggregation approach. *IEEE Access* **2020**, *8*, 23920–23935. [CrossRef]
29. Li, C.; Zhang, Y.; Luo, Y. A federated learning-based edge caching approach for mobile edge computing-enabled intelligent connected vehicles. *IEEE Trans. Intell. Transp. Syst.* **2022**, *24*, 3360–3369. [CrossRef]
30. Xiao, H.; Rasul, K.; Vollgraf, R. Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. *arXiv* **2017**, arXiv:1708.07747.
31. Recht, B.; Roelofs, R.; Schmidt, L.; Shankar, V. Do cifar-10 classifiers generalize to cifar-10? *arXiv* **2018**, arXiv:1806.00451.
32. Yang, Z.; Wang, R.; Wu, D.; Wang, H.; Song, H.; Ma, X. Local trajectory privacy protection in 5G enabled industrial intelligent logistics. *IEEE Trans. Ind. Inform.* **2021**, *18*, 2868–2876. [CrossRef]
33. Yoshida, N.; Nishio, T.; Morikura, M.; Yamamoto, K.; Yonetani, R. Hybrid-FL for wireless networks: Cooperative learning mechanism using non-IID data. In Proceedings of the ICC 2020—2020 IEEE International Conference on Communications (ICC), Dublin, Ireland, 7–11 June 2020; pp. 1–7.
34. McMahan, B.; Moore, E.; Ramage, D.; Hampson, S.; y Arcas, B.A. Communication-efficient learning of deep networks from decentralized data. In Proceedings of the Artificial Intelligence and Statistics, PMLR, Fort Lauderdale, FL, USA, 20–22 April 2017; pp. 1273–1282.
35. Lu, X.; Liao, Y.; Liu, C.; Lio, P.; Hui, P. Heterogeneous model fusion federated learning mechanism based on model mapping. *IEEE Internet Things J.* **2021**, *9*, 6058–6068. [CrossRef]